The Journal of Open Source Software

AutoUncertainties: A Python Package for Uncertainty Propagation

Varchas Gopalaswamy [1*] **and Ethan Mentzer** [1*]

1 Laboratory for Laser Energetics, Rochester, USA * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 March 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Propagation of uncertainties is of great utility in the experimental sciences. While the rules of (linear) uncertainty propagation are simple, managing many variables with uncertainty information can quickly become complicated in large scientific software stacks, and require keeping track of many variables and implementing custom error propagation rules for each mathematical operator. The Python package `AutoUncertainties`, described here, provides a solution to this problem.

Statement of Need

`AutoUncertainties` is Python package for uncertainty propagation. It provides a drop-in mechanism to add uncertainty information to Python scalar and `NumPy` (Harris et al., 2020) array variables. It implements manual propagation rules for the Python dunder math methods, and uses automatic differentiation via `JAX` (Bradbury et al., 2018) to propagate uncertainties for most `NumPy` methods applied to both scalar and `NumPy` array variables. In doing so, it eliminates the need for carrying around additional uncertainty variables, needing to implement custom propagation rules for any `NumPy` operator with a gradient rule implemented by `JAX`, and in most cases requires minimal modification to existing code, typically only when uncertainties are attached to central values.

Prior Work

To the author's knowledge, the only existing error propagation library in Python is the `uncertainties` (Lebigot et al., 2024) package, which inspired the current work. While extremely useful, the `uncertainties` package relies on hand-implemented rules and functions for uncertainty propagation of array and scalar data. While this is transparent for the intrinsic dunder methods such as `__add__`, it becomes problematic for advanced mathematical operators. For instance, calculating the uncertainty propagation due to the cosine requires the import of separate math libraries

```
import numpy as np
from uncertainties import unumpy, ufloat
arr = np.array([ufloat(1, 0.1), ufloat(2, 0.002)])
unumpy.cos(arr)  # calculation succeeds
```

rather than being able to use `NumPy` directly.

```
import numpy as np
from uncertainties import ufloat
```

```
arr = np.array([ufloat(1, 0.1), ufloat(2, 0.002)])
np.cos(arr)  # raises an exception
```

Implementation

Linear uncertainty propagation of a function $f(x) : \mathbb{R}^n \to \mathbb{R}^m$ can be computed via the simple rule

$$\delta f_j(x)^2 = \left(\frac{\partial f_j}{\partial x_i}(x)\, \delta x_i \right)^2 .$$

To compute $\frac{\partial f_j}{\partial x_i}$ for arbitrary f, the implementation in `AutoUncertainties` relies on automatic differentiaion provided by `JAX`. Calls to any `NumPy` array function or universal function (ufunc) are intercepted via the `__array_function__` and `__array_ufunc__` mechanism, and dispatched to a `NumPy` wrapper routine that computes the Jacobian matrix via `jax.jacfwd`.

The user API for the `Uncertainty` object exposes a number of properties and methods, of which some of the most important are:

- `value -> float`: The central value of the object.
- `error -> float`: The error of the object.
- `relative -> float`: The relative error (i.e. error / value) of the object.
- `plus_minus(self, err: float) -> Uncertainty`: Adds error (in quadrature).
- `from_sequence(cls, seq: Sequence) -> VectorUncertainty`: Constructs an array `Uncertainty` object from some existing sequence.

These attributes and methods can be used in the following manner:

```
from auto_uncertainties import Uncertainty
u1 = Uncertainty(5.25, 0.75)
u2 = Uncertainty(1.85, 0.4)

print(u1)                # 5.25 +/- 0.75
print(u1.value)          # 5.25
print(u1.error)          # 0.75
print(u1.relative)       # 0.142857
print(u1.plus_minus(0.5))  # 5.25 +/- 0.901388

seq = Uncertainty.from_sequence([u1, u2])
print(seq)  # [5.25 +/- 0.75, 1.85 +/- 0.4]
```

To extract errors / central values from arbitrary objects, the accessors `nominal_values` and `std_devs` are provided. These functions return:

- The central values and errors, respectively, if the input is an `Uncertainty` object.
- The input and zero if the input is any other kind of object.

`Uncertainty` objects are displayed using rounding rules based on the uncertainty, i.e.,

- Error to 2 significant digits.
- Central value to first signficant digit of error, or two significant figures (whichever is more significant digits).

This behavior can be toggled using `set_display_rounding`:

```
from auto_uncertainties import set_display_rounding
set_display_rounding(False)
```

DRAFT

Calling `__array__`, whether via `numpy.array` or any other method, will by default issue a warning, and convert the `Uncertainty` object into an equivalent array of its nominal values, stripping all error information. To prevent this behavior, the `set_downcast_error` function can be called so that an exception is raised instead:

```
from auto_uncertainties import set_downcast_error
set_downcast_error(True)
```

Support for Pint

`AutoUncertainties` provides some support for working with objects from the `Pint` package (Grecco & Chéron, 2025). For example, `Uncertainty` objects can be instantiated from `pint.Quantity` objects, and then automatically wrapped into new `pint.Quantity` objects via the `from_quantities` method. This guarantees that unit information is preserved when moving between `Uncertainty` objects and `pint.Quantity` objects.

```
from auto_uncertainties import Uncertainty
from pint import Quantity

val = Quantity(2.24, 'kg')
err = Quantity(0.208, 'kg')
new_quantity = Uncertainty.from_quantities(val, err)

print(new_quantity)        # 2.24 +/- 0.208 kilogram
print(type(new_quantity))  # <class 'pint.Quantity'>
```

Pandas

Support for `pandas` (The pandas development team, 2024) via the `ExtensionArray` mechanism is largely functional. Upcoming aditions to `AutoUncertainties` will further improve compatibility.

Current Limitations and Future Work

Dependent Random Variables

To simplify operations on `Uncertainty` objects, `AutoUncertainties` assumes all variables are independent and normally distributed. This means that, in the case where the programmer assumes dependence between two or more `Uncertainty` objects, unexpected and counter-intuitive behavior may arise. Two examples of this behavior follow.

Subtracting Equivalent Uncertainties

Subtracting an `Uncertainty` from itself will not result in a standard devation of zero, as demonstrated in the following example:

```
x = Uncertainty(5.0, 0.5)
print(x - x)  # 0 +/- 0.707107
```

Mean Error Propagation

When multiplying a vector by a scalar `Uncertainty` object, each component of the resulting vector is assumed to be a multivariate normal distribution with no covariance, which may not be the desired behavior. For instance, taking the mean of such a vector will return an `Uncertainty` object with an unexpectedly small standard deviation.

DRAFT

```
u = Uncertainty(5.0, 0.5)
arr = np.ones(10) * 10
result = np.mean(u * arr)  # 50 +/- 1.58114, rather than 50 +/- 5 as expected
```

To obtain the uncertainty corresponding to the case where each element of the array is fully correlated, two workaround techniques can be used:

1. Separate the central value from the relative error, multiply the vector by the central value, take the mean of the resulting vector, and then multiply by the previously stored relative error.

   ```
   u = Uncertainty(5.0, 0.5)
   scale_error = Uncertainty(1, u.relative)  # collect relative error
   scale_value = u.value                     # collect central value

   arr = np.ones(10) * 10
   result = np.mean(scale_value * arr) * scale_error  # 50 +/- 5
   ```

2. Take the mean of the vector, and then multiply by the `Uncertainty`:

   ```
   u = Uncertainty(5.0, 0.5)
   arr = np.ones(10) * 10
   result = u * np.mean(arr)  # 50 +/- 5
   ```

These workarounds are nevertheless cumbersome, and cause `AutoUncertainties` to fall somewhat short of the original goals of automated error propagation. In principle, this could be addressed by storing a full computational graph of the result of chained operations, similar to what is done in `uncertainties`. However, the complexity of such a system places it out of scope for `AutoUncertainties` at this time.

Typing System

Type hinting is employed throughout `AutoUncertainties` to aid static analysis of the package. At this time, however, many typing inconsistencies can be detected by static type enforcement tools like Mypy and Pyright. Future improvements to `AutoUncertainties` will likely include typing adjustments to the code, in order to avoid subtle bugs.

Further Information

Additional API information and usage examples can be found on the documentation website. All source code for the project is stored and maintained on the `AutoUncertainties` GitHub repository, where contributions, suggestions, and bug reports are welcome.

Acknowledgements

This material is based upon work supported by the Department of Energy [National Nuclear Security Administration] University of Rochester "National Inertial Confinement Fusion Program" under Award Number(s) DE-NA0004144, and Department of Energy [Office of Fusion Energy Sciences] University of Rochester "Applications of Machine Learning and Data Science to predict, design and improve laser-fusion implosions for inertial fusion energy" under Award Number(s) DE-SC0024381.

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned

DRAFT

rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

References

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.5.1). https://github.com/jax-ml/jax

Grecco, H. E., & Chéron, J. (2025). Pint: Makes units easy. In *GitHub repository*. GitHub. https://github.com/hgrecco/pint

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Lebigot, E. O., Newville, M., Deil, C., Davar, G., Shanks, W., Savara, A., Yurchak, R., andrewgsavage, ep12, Gerber, J., Pieters, M., Romano, P., Riley, W., Baldwin, A., Abel, B., Burr, C., op3, Eendebak, P., & Cladé, P. (2024). *Lmfit/uncertainties: 3.2.2* (Version 3.2.2). Zenodo. https://doi.org/10.5281/zenodo.12682754

The pandas development team. (2024). *Pandas-dev/pandas: pandas* (Version v2.2.3). Zenodo. https://doi.org/10.5281/zenodo.13819579

DRAFT